Universidad Tecnológica Nacional

Facultad Regional Rosario

Desarrollo de Software

Trabajo Práctico Integrador

City Garage Rental

Docentes:

- Adrián Meca
- Lucas Luna

Comisión: 304

Año de cursado: 2024

Integrante:

Brian Nahuel Ortigosa - 47350

Índice

Introducción **2**
Link repositorio Github **3**
Diagrama Entidad Relación (DER) **3**
Video explicación del sistema **3**
DEPLOY **3**
Instrucciones de instalación **4**
- **Requisitos previos** **4**
- **Paso a paso** **4**

Minutas de reunión **6**
Tracking Features **11**
Tracking de Bugs / Issues **12**
Documentación de API **13**
Testing **13**
- **Herramientas de testing** **13**
- **Tests de Backend** **14**
  - Tests Unitarios — GarageRepository 14
  - Tests de Integración — Endpoints HTTP 16
- **Tests de Frontend** **18**
  - Tests Unitarios — GaragesService 18
  - Tests E2E — LoginComponent 19

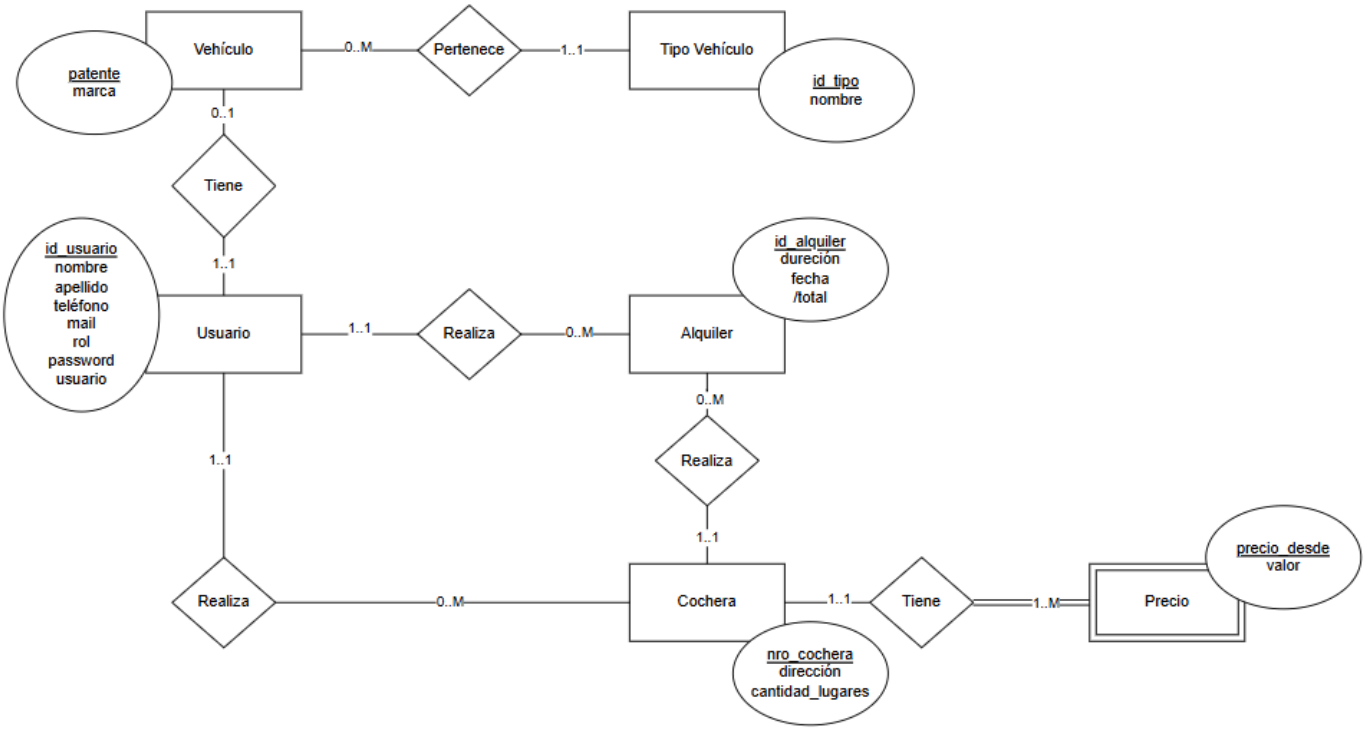
Introducción

City Garage Rental es una aplicación web full-stack diseñada para facilitar el alquiler de cocheras en entornos urbanos. La plataforma conecta a propietarios de garajes con usuarios que necesitan un espacio de estacionamiento, permitiendo explorar disponibilidad en tiempo real y concretar reservas directamente desde el sitio, de forma segura y sin intermediarios.
El sistema cuenta con autenticación de usuarios, gestión de espacios por parte de los propietarios y un flujo de pago integrado, todo dentro de una interfaz moderna y adaptable a dispositivos móviles.

Link repositorio Github

https://github.com/Dragtrop/Repositorio-TP-Backend.git

Diagrama Entidad Relación (DER)



Video explicación del sistema

En el siguiente link a YouTube se encuentra el video con una breve explicación de las funcionalidades del sistema y el funcionamiento del código.

Link: https://youtu.be/8VsQKm3FWAk

DEPLOY

Link de la aplicación: repositorio-tp-backend-production.up.railway.app

Instrucciones de instalación

Requisitos previos

Antes de levantar el proyecto, es necesario tener instalado en tu computadora las siguientes herramientas:

Herramienta	Versión recomendada	Descarga
Node.js	20.x o superior	nodejs.org
pnpm	9.1.1	npm install -g pnpm
MySQL	8.x	mysql.com
Angular CLI	17.x	npm install -g @angular/cli@17

Paso a paso

1. Clonar el repositorio

a. Clonar el repositorio en tu computadora con los comandos:

```
git clone https://github.com/Dragtrop/Repositorio-TP-Backend.git
cd <carpeta>
```

2. Configurar la base de datos

a. Abrir MySQL y crear una base de datos llamada `cocheras`.
b. Copiar el archivo que se encuentra en /backend/src/shared llamado schema.sql en MySQL workbench y ejecutar el script.

3. Configurar las variables de entorno del backend

a. Dentro de la carpeta `/backend`, crear un archivo llamado `.env`
b. Copiar el siguiente formato y completar los datos:

```
DB_HOST=localhost
DB_USER=tu_usuario_mysql
```

```
DB_PASSWORD=tu_contraseña_mysql
DB_NAME=cocheras
PORT=3000
```

4. Instalar dependencias del backend

a. Instalar las dependencias con estos comandos

```
cd ../backend
pnpm install
```

5. Iniciar el backend

a. Levantar el backend con el siguiente comando:

```
pnpm start:dev
```

Nota: El servidor va a correr en http://localhost:3000. Es importante levantar el backend antes que el frontend ya que este último consume la API que está en el backend.

6. Instalar dependencias del frontend

a. Instalar las dependencias con estos comandos:

```
cd ../frontend
pnpm install
```

7. Iniciar el frontend

a. Levantar el frontned con este comando:

```
ng serve
```

Nota: La aplicación va a estar disponible en http://localhost:4200

8. Abrir túneles para pagos

Nota: Los túneles de Cloudflare son necesarios porque MercadoPago necesita acceder a URLs públicas para redirigir al usuario después del pago y para enviar notificaciones (webhooks) al backend sobre el estado de la transacción.

a. Abrir túneles del backend y del front end con estos comandos en dos nuevas terminales:

```
npx cloudflared tunnel --url http://localhost:4200

npx cloudflared tunnel --url http://localhost:3000
```

b. Pegar el URL que devuelve cloudflared en el archivo .env del backend:

FRONTEND_URL=tomato-snake-pvc-nothing.trycloudflare.com/principal
BACKEND_URL=tomato-snake-pvc-nothing.trycloudflare.com

c. Volver a levantar el backend y el frontend con estos comandos:

pnpm start:dev

ng serve --disable-host-check

Nota: La opción --disable-host-check es necesaria porque el frontend deja de ser accedido únicamente desde localhost y pasa a utilizarse también mediante la URL pública generada por Cloudflare. Angular bloquea hosts no reconocidos por seguridad.

Minutas de reunión

Minuta 1 – Inicio del proyecto

Fecha: 20/06/2024
Objetivo: Definir la base del sistema y tecnologías a utilizar

Temas tratados:

- Definición del alcance del sistema
- Selección de tecnologías (Node.js, MySQL, Angular)
- Estructuración inicial del repositorio

Decisiones:

- Uso de arquitectura cliente-servidor
- Base de datos relacional (MySQL)
- Separación en frontend y backend

Minuta 2 – Desarrollo de backend

Fecha: 27/06/2024

Objetivo: Implementar persistencia y lógica de negocio

Temas tratados:

- Integración con MySQL
- Implementación de CRUD de vehículos
- Configuración de repositorios

Decisiones:

- Uso de patrón Repository
- Manejo de consultas mediante pool de conexiones

Problemas detectados:

- Errores en configuración de MySQL

Minuta 3 – Integración frontend-backend

Fecha: 25/07/2024

Objetivo: Conectar el frontend con el backend

Temas tratados:

- Consumo de API desde Angular
- Problemas con GET ALL en frontend

Problemas detectados:

- Error en obtención de datos
- Fallas en sincronización de endpoints

Decisiones:

- Revisar endpoints y servicios Angular

Minuta 4 – Implementación completa de CRUD

Fecha: 14/09/2024

Objetivo: Finalizar operaciones CRUD

Temas tratados:

- CRUD completo en frontend y backend
- Integración de Location y VehicleType

Decisiones:

- Validar datos desde frontend y backend
- Mejorar estructura de componentes

Minuta 5 – Implementación de autenticación

Fecha: 11/11/2024

Objetivo: Implementar login de usuarios

Temas tratados:

- Login en frontend y backend
- Manejo de usuarios

Decisiones:

- Validación de credenciales en backend
- Manejo de sesión/autenticación

Minuta 6 – Defensa para regularidad

Fecha: 21/11/2024
Tipo de reunión: Evaluación / Defensa del proyecto
Objetivo: Presentar el avance del sistema y validar la regularidad de la materia

Temas tratados:

- Presentación general del sistema *City Garage Rental Web Application*
- Demostración del funcionamiento del login (frontend y backend)
- Exposición de CRUDs implementados
- Explicación de la arquitectura (Angular + Node.js + MySQL)
- Estado del módulo de perfil y primeras funcionalidades de alquiler

Observaciones recibidas:

- Se sugirió mejorar la organización del código en el frontend
  Se recomendó implementar validaciones más robustas
- Se indicó la necesidad de incorporar mayor control de acceso (seguridad / protección de rutas)
- Se mencionó la posibilidad de mejorar la experiencia de usuario (UI/UX)

Decisiones:

- Continuar con el desarrollo del módulo de alquiler
- Incorporar mejoras en validación de datos
- Planificar la implementación de seguridad en rutas

Conclusiones:

- El sistema contaba con una base funcional sólida (CRUDs + login)
- Se identificaron áreas de mejora principalmente en seguridad y experiencia de usuario
- Se definió una segunda etapa orientada a robustecer el sistema

Minuta 7 – Nuevas funcionalidades y mejoras

Fecha: 07/11/2024

Objetivo: Ampliar funcionalidades del sistema

Temas tratados:

- Implementación de perfil de usuario
- Inicio del módulo de alquiler

Decisiones:

- Separación de módulos por funcionalidad

Minuta 8 - Consulta con profesor

Fecha: 28/04/2025
Tipo de reunión: Consulta técnica / seguimiento
Objetivo: Revisar el avance posterior a la defensa y resolver dudas técnicas

Temas tratados:

- Estado del sistema luego de la defensa
- Dudas sobre mejoras en frontend (responsive design)
- Problemas en consultas SQL y estructura de datos
- Organización del código y buenas prácticas

Problemas planteados:

- Dificultades en adaptación a dispositivos móviles
- Inconsistencias en consultas SQL
- Necesidad de mejorar la mantenibilidad del código

Recomendaciones recibidas:

- Aplicar diseño responsive (especialmente en layout: sidebar y header)
- Optimizar consultas SQL
- Modularizar mejor el frontend
- Incorporar buenas prácticas en el manejo del estado

Decisiones:

- Iniciar mejoras en la versión mobile del frontend
- Refinar lógica de base de datos
- Continuar evolución del sistema hacia una versión más estable

Minuta 9 – Adaptación mobile y mejoras

Fecha: 15/07/2025

Objetivo: Adaptar interfaz a dispositivos móviles

Temas tratados:

- Problemas de layout (sidebar y header)
- Responsive design

Problemas detectados:

- Sidebar no visible en mobile
- Mala disposición de componentes

Minuta 10 – Etapa final

Fecha: 26/02/2026

Objetivo: Finalizar funcionalidades principales

Temas tratados:

- CRUD de cocheras
- Ajustes finales del sistema

Decisiones:

- Consolidar funcionalidades principales

Minuta 11 – Implementación y ejecución de tests

Fecha: 01/03/2026
Tipo de reunión: Desarrollo / Validación de calidad
Objetivo: Implementar y ejecutar pruebas automatizadas para validar el correcto funcionamiento del sistema

Temas tratados:

- Implementación de testing en backend utilizando Jest y Supertest
- Simulación de repositorios mediante mocks para aislar la lógica de negocio
- Validación de endpoints REST del módulo de garajes
- Testing de repositorio (acceso a datos y consultas SQL)
- Implementación de testing en frontend con Angular Testing Module
- Validación de servicios HTTP (GaragesService)
- Testing de componente de login (flujo completo)

- Simulación de respuestas HTTP con HttpTestingController

Problemas detectados:

- Casos donde no se validaban correctamente errores del backend
- Inconsistencias en respuestas HTTP (estructura de datos)

Decisiones:

- Implementar mocks para aislar el testing del repositorio y middlewares
- Validar tanto casos exitosos como errores
- Asegurar cobertura de endpoints críticos
- Incorporar testing en frontend para servicios y componentes clave

Tracking Features

Feature	Estado	Descripción
Configuración inicial del proyecto	TRUE	Setup de frontend y backend
Integración con MySQL	TRUE	Conexión y persistencia de datos
CRUD Vehículos	TRUE	Alta, baja, modificación y consulta
CRUD Vehicle Types	TRUE	Gestión de tipos de vehículos
CRUD Location	TRUE	Gestión de ubicaciones
CRUD Cocheras	TRUE	Gestión de cocheras
CRUD Alquileres	TRUE	Gestión de alquileres
CRUD Hitorial - Precio	TRUE	Gestión de los precios históricos de los garajes
CRUD Usuarios	TRUE	Gestión de los usuarios
Registro de usuario	TRUE	Usuarios crear una cuenta en el sistema
Dashboard	TRUE	Vista general del sistema al usuario una vez autenticado
Filtros	TRUE	Permite refinar la visualización de datos en los listados
Paginación	TRUE	Implementa la división de grandes volúmenes de datos en páginas
Login de usuarios	TRUE	Autenticación frontend y backend
Perfil de usuario	TRUE	Visualización y gestión
CU - Usuario Realiza Alquiler	TRUE	Lógica de alquiler
Responsive mobile	TRUE	Adaptación de UI

Historial de precios	TRUE	Registro de cambios de precios
Protección de rutas	TRUE	Seguridad en frontend/backend
Módulo garajes	TRUE	Operaciones relacionadas a las cocheras
Módulo vehículo	TRUE	Gestión de vehículos asociados a los usuarios
Baja lógica Usuarios	TRUE	Desactivación de usuarios sin eliminación física de la base de datos
Validar formato de patente	TRUE	Validar que la patente ingresa cumpla con el formato usado en la Argentina (viejo / nuevo)
CU - Pagar alquiler	TRUE	Lógica del pago de los alquileres

Tracking de Bugs / Issues

Bug / Issue	Estado	Descripción
Error conexión MySQL	TRUE	Problemas iniciales de configuración
Error GET ALL frontend	TRUE	No se mostraban datos correctamente
Error dist/server.js	TRUE	Problema al levantar backend
Problemas en modificación de cocheras	TRUE	Fallas en actualización
Sidebar mal posicionado	TRUE	Problema de layout
Sidebar no visible en mobile	TRUE	Falta implementación responsive
Expansión incorrecta de tarjetas	TRUE	Bug en UI
Problemas de login	TRUE	Validación incorrecta
Errores SQL	TRUE	Ajustes en queries
Sin protección de rutas	TRUE	Falta implementación de seguridad
Falta entidad débil	TRUE	falta implementar entidad débil de cochera
Error en el breadcrumbs de detalle-alquiler	TRUE	Muestra el id del alquiler en vez de "Detalle alquiler"
Validación de alquiler	TRUE	No valida si el usuario tiene vehículo asignado
Problemas con el registro	TRUE	Se puede registrar un admin (no debería)
Mala referenciación a la CP de garage	TRUE	Se busca por id en vez de por nroGarage
Error en filtros	TRUE	Se filtra por página, debería filtrar por toda la colección
Error al mostrar historial de precios	TRUE	Falta arreglar el html y css del historial de precios en el módulo garajes

Error al asignar vehículo al usuario	TRUE	Siempre se asigna id = 1 a los usuarios
Problemas con node_modules	TRUE	Inclusión/eliminación incorrecta en repositorio
Error en endpoint findOne	TRUE	Problemas en obtención de registros individuales
Problemas en CRUD Usuario	TRUE	Múltiples correcciones necesarias en implementación
Datos no persistidos correctamente	TRUE	Fallas en guardado en base de datos
Errores en layout responsive	TRUE	Elementos desalineados en distintas resoluciones
Problemas en navegación	TRUE	Rutas no redirige correctamente (childrens del principal)
Falta de validación en formularios	TRUE	Inputs permiten datos inválidos
Problemas en estructura del proyecto	TRUE	Organización inicial poco clara
Problemas en relaciones de base de datos	TRUE	Relaciones mal definidas entre entidades
Datos duplicados	FALSE	Falta de control de unicidad
Problemas en carga inicial (dashboard)	TRUE	Datos no cargan correctamente al iniciar
Problemas en UI de perfil	TRUE	Visualización inconsistente
Admin desactiva Admin	TRUE	No se debería poder desactivar un admin desde el panel de usuarios
No se puede eliminar garaje	TRUE	No deja eliminar el garaje que tenga un alquiler hecho debido a foreign key de alquileres.
Header te hace logout	TRUE	Cuando apretas la imagen para volver a la página principal te hace logout.
Tipo de dato Teléfono	TRUE	El tipo de dato no es el correcto. Requiere BIGINT()

Documentación de API

La API está documentada en formato YAML y se puede visualizar en el siguiente link usando la herramienta gratuita Swagger:
https://app.swaggerhub.com/apis-docs/brianchu44organizati/garage-app-api/1.0.0?view=uiDocs

Testing

Herramientas de testing

Herramienta	Descripción	Capa
Jest 29	Framework de testing JS/TS. Soporta mocks, spies y assertions.	Backend
ts-jest	Transforma TypeScript para Jest sin compilacion previa.	Backend
Supertest	Realiza peticiones HTTP reales contra Express en memoria.	Backend
@jest/globals	Importación explícita de funciones Jest para proyectos ESModules.	Backend
Karma + Jasmine	Runner y framework de tests incluido por defecto en Angular CLI.	Frontend
HttpClientTestingModule	Reemplaza HttpClient por uno controlable para simular respuestas.	Frontend
HttpTestingController	Intercepta y verifica requests HTTP en los tests.	Frontend

Tests de Backend

Ejecución: pnpm test

Tests Unitarios — GarageRepository

Archivo: tests/unit/garage.repository.test.ts

Verifican cada método del GarageRepository de forma completamente aislada. Se mockea el pool de MySQL con jest.unstable_mockModule (API correcta para ES Modules), permitiendo controlar exactamente que devuelve cada query sin necesitar base de datos real.

Casos de prueba

Método	Caso de prueba	Resultado esperado
findOne	Devuelve un garage si existe	Objeto Garage
findOne	Devuelve undefined si no existe	undefined
findOne	Lanza error si el ID no es válido	Error lanzado
findAll	Devuelve lista de garajes	Array de Garajes
findAll	Devuelve lista vacía si no hay garajes	[]
findByOwner	Devuelve garajes de un dueño	Array de Garajes
findByOwner	Lista vacía si el dueño no tiene garajes	[]
findAllPaginated	Devuelve datos paginados con total	{ data, total }
add	Inserta garaje y devuelve el creado	Garage con ID asignado
delete	Elimina garaje existente y lo devuelve	Garage eliminado
delete	Devuelve undefined si no existe	undefined

deleteByNro	Elimina por nroGarage y devuelve garage	Garage eliminado
deleteByNro	Devuelve undefined si no existe	undefined
findByNroGarage	Devuelve garaje por nroGarage	Objeto Garage
findByNroGarage	Devuelve undefined si no existe	undefined
updateCantLugares	Actualiza cantLugares correctamente	Garage actualizado
getHistorialPrecios	Devuelve historial de precios	Array de historial

Total: 17 tests unitarios — todos pasando.

Tests de Integración — Endpoints HTTP

Archivo: tests/unit/auth.middleware.test.ts

Verifican el flujo completo: ruta → middleware → controller. Supertest realiza peticiones HTTP reales contra la app Express sin servidor ni DB. El repository y los middlewares de autenticacion son mockeados para aislar la lógica del controller.

Estrategia de mocks

- GarageRepository: todos sus métodos reemplazados por jest.fn().
- authMiddleware: inyecta req.user directamente, simulando usuario autenticado con rol Dueno.
- roleMiddleware: ejecuta la lógica real de verificación de rol (req.user.Rol).

Casos de prueba

Endpoint	Caso de prueba	Codigo HTTP

GET /todos	Devuelve lista de garajes	200 OK
GET /todos	Error de base de datos	500 Internal Server Error
GET /:id	Devuelve un garage por ID	200 OK
GET /:id	Garage no encontrado	404 Not Found
GET /:id	ID invalido (no numerico)	400 Bad Request
POST /	Crea un garage correctamente	201 Created
PUT /:id	Actualiza un garage correctamente	200 OK
PUT /:id	Usuario no es el dueño	403 Forbidden
PUT /:id	Garage no encontrado	404 Not Found
DELETE /:id	Elimina un garage correctamente	200 OK
DELETE /:id	Usuario no es el dueño	403 Forbidden
DELETE /:id	Garage no encontrado	404 Not Found
GET /mis-garages	Devuelve garajes del dueño autenticado	200 OK
GET /historial/:nro	Devuelve historial de precios	200 OK
PUT /:id/alquilar	Alquila con lugares disponibles	200 OK

PUT /:id/alquilar	Garage no encontrado	404 Not Found
PUT /:id/alquilar	Sin lugares disponibles	400 Bad Request

Total: 17 tests de integración — todos pasando.

```
PS C:\Users\Usuario\Desktop\UTN\Desarrollo de Software\Nuevo_Trabajo\Repositorio-TP-Backend\backend> pnpm test
> backend@1.0.0 test C:\Users\Usuario\Desktop\UTN\Desarrollo de Software\Nuevo_Trabajo\Repositorio-TP-Backend\backend
> node --experimental-vm-modules ./node_modules/jest/bin/jest.js

(node:13768) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/unit/garage.respository.test.ts
(node:17196) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
 PASS  tests/unit/auth.middleware.test.ts (11.23 s)
```

```
Test Suites: 2 passed, 2 total
Tests:       34 passed, 34 total
Snapshots:   0 total
Time:        13.409 s
Ran all test suites.
```

Tests de Frontend

Tests Unitarios — GaragesService

Archivo: src/app/services/garages.service.spec.ts

Ejecución: ng test --include="**/garages.service.spec.ts" --watch=false

Verifican que GaragesService realiza correctamente cada petición HTTP. Se usa HttpClientTestingModule para interceptar requests y HttpTestingController para verificar metodo, URL y body. afterEach() llama a httpMock.verify() para asegurar que no queden requests sin resolver.

Casos de prueba

Método	Verificación	HTTP
ConsultarGarage	GET con parámetros page y limit	GET

ConsultarGarageTodos	GET a /todos devuelve lista completa	GET
getMisGarages	GET a /mis-garages devuelve garajes del dueño	GET
getgarage	GET por ID y extracción del campo data	GET
addgarage	POST con el body correcto	POST
editgarage	PUT con ID y datos actualizados	PUT
deletegarage	DELETE por ID	DELETE
alquilarGarage	PUT a /alquilar con cantidad en el body	PUT
getHistorial	GET a /historial/:nroGarage	GET
(instancia)	El servicio se crea correctamente	—

Total: 10 tests unitarios — todos pasando.

```
PS C:\Users\Usuario\Desktop\UTN\Desarrollo de Software\Nuevo_Trabajo\Repositorio-TP-Backend\frontend> ng test --include="**/garages.service.spec.ts" --watch=false
✓ Browser application bundle generation complete.
18 03 2026 10:13:59.267:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
18 03 2026 10:13:59.270:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
18 03 2026 10:13:59.291:INFO [launcher]: Starting browser Chrome
18 03 2026 10:14:04.779:INFO [Chrome 146.0.0.0 (Windows 10)]: Connected on socket jjICzC-0zd8EoyYIAAAB with id 12209051
Chrome 146.0.0.0 (Windows 10): Executed 10 of 10 SUCCESS (0.425 secs / 0.034 secs)
TOTAL: 10 SUCCESS
```

Tests E2E — LoginComponent

Archivo: src/app/components/login/login.component.spec.ts

Ejecución: ng test --include="**/login.component.spec.ts" --watch=false

Simulan la interacción completa del usuario con el componente de login. AuthService y Router son reemplazados por spies de Jasmine. Se usa fakeAsync + tick() para resolver observables asíncronos y verificar la navegación post-login.

Configuración del entorno

- AuthService: reemplazado por jasmine.createSpyObj con el método login mockeado.
- Router: reemplazado por jasmine.createSpyObj con el método navigate mockeado.
- provideRouter([]): provee el router sin rutas reales para evitar errores de inicialización.

Casos de prueba

Caso de prueba	Tipo	Resultado esperado
El componente se crea correctamente	Unitario	component.toBeTruthy()
Usuario y password vacíos al inicio	Estado inicial	valores en ''
Navega a /principal/dashboard tras login exitoso	E2E	router.navigate llamado
No navega si el login falla	E2E	navigate no llamado
Llama a AuthService con valores del formulario	E2E	authService.login(usuario, pass)
Lee inputs del template y hace submit	E2E DOM	authService.login llamado

Total: 6 tests E2E — todos pasando.

```
PS C:\Users\Usuario\Desktop\UTN\Desarrollo de Software\Nuevo_Trabajo\Repositorio-TP-Backend\frontend> ng test --include="**/login.component.spec.ts" --watch=false
✓ Browser application bundle generation complete.
18 03 2026 10:09:01.447:INFO [karma-server]: Karma v6.4.4 server started at http://localhost:9876/
18 03 2026 10:09:01.450:INFO [launcher]: Launching browsers Chrome with concurrency unlimited
18 03 2026 10:09:01.472:INFO [launcher]: Starting browser Chrome
18 03 2026 10:09:05.164:INFO [Chrome 146.0.0.0 (Windows 10)]: Connected on socket 3tqSh4r1S0s9gpjSAAAB with id 67901673
```

```
Chrome 146.0.0.0 (Windows 10): Executed 6 of 6 SUCCESS (0.087 secs / 0.079 secs)
TOTAL: 6 SUCCESS
```